

FIAP — Faculdade de Informática e Administração Paulista

15SOAT — Pós-Graduação em Arquitetura de Software



Tech Challenge — Fase 2

PytStop — Plataforma de Gestão de Ordens de Serviço

Documento de Entrega

João Amaral — RM373448 · Allan Aurélio — RM372116 · Carlos Silva —
RM374191

Guilherme Sousa — RM373609 · Nicolas Gerbi — RM372644

São Paulo — 2026

Documento de Entrega — Tech Challenge Fase 2

| **Versão:** 1.4 — Julho/2026.

Documento de entrega da Fase 2 do Tech Challenge da Pós-Graduação em Arquitetura de Software (FIAP). O conteúdo cobre os itens exigidos pelo enunciado da fase: identificação do grupo, link do repositório (compartilhado com o avaliador), desenho da arquitetura, instruções de execução e deploy, link da collection das APIs e link do vídeo de demonstração.

Como ler este documento

O repositório é a fonte de verdade da entrega. Os artefatos exigidos pela fase — código refatorado com Clean Architecture, Dockerfile e docker-compose revisados, manifests Kubernetes em /k8s, scripts Terraform em /infra, pipeline de CI/CD e README atualizado — estão versionados no próprio projeto. Os links abaixo apontam diretamente para esses arquivos no GitHub (branch main), navegáveis pela UI nativa do GitHub com o avaliador adicionado como colaborador. O desenho da arquitetura é modelado em Mermaid, renderizado nativamente pelo GitHub; a fonte única do diagrama é a [RFC-002 §3](#), e ele é replicado no [README](#) e na seção 7 deste documento.

A opção por documentação textual e versionada segue a fase 1: o projeto é AI-first, e Markdown + Mermaid permitem manutenção por agentes de IA sem prejuízo da leitura humana. As decisões da fase estão registradas em ADRs (015–024) e consolidadas na RFC-002; a rastreabilidade requisito → implementação → evidência está na seção 6.

1. Identificação do Grupo

Campo	Valor
Nome do grupo	PytStop
Turma	15SOAT — Pós-Graduação em Arquitetura de Software (FIAP)

Participantes

Nome	RM	Discord
João Amaral	RM373448	joao_13997
Allan Aurélio	RM372116	all66_
Carlos Silva	RM374191	carlossilva156
Guilherme Sousa	RM373609	romen0
Nicolas Gerbi	RM372644	sethiiz_gerbi

2. Link do Repositório

Repositório privado no GitHub, compartilhado com soat-architecture conforme exigido pelo enunciado. A fase 2 continua no mesmo histórico da fase 1: o repositório preserva os 118 commits do MVP ([PR #11](#)) e evolui a partir deles.

Recurso	URL
Repositório	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2
README (arquitetura, execução local, deploy K8s, Terraform)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/README.md
Dockerfile	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/Dockerfile
docker-compose.yml	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docker-compose.yml
Manifests Kubernetes (/k8s)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/tree/main/k8s
Scripts Terraform (/infra)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/tree/main/infra
Pipeline de CI (herdada da fase 1)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/.github/workflows/ci.yml
Pipeline de CD (build de imagem + deploy + smoke test)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/.github/workflows/cd.yml
Collection das APIs (Postman, gerada do OpenAPI)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/entrega/fase2/postman_collection.json

A collection foi gerada a partir do contrato OpenAPI vivo da aplicação (46 requisições agrupadas por tag) e é executável de ponta a ponta — validada com newman contra o cluster ([PR #157](#)). O request de decisão externa de orçamento traz um **pre-request script que assina a requisição com HMAC** (X-Webhook-Signature + X-Webhook-Timestamp, espelhando `webhook_signature.py`). O Swagger UI em `/docs` permanece a referência interativa — instruções de acesso no README.

Execuções verdes do CD na main

O pipeline de CD provisiona um cluster kind (Kubernetes in Docker) efêmero no runner via Terraform, publica a imagem no GHCR (GitHub Container Registry) com tag imutável por SHA e aplica os manifests com smoke test ao final ([ADR-019](#)):

Execução	Conteúdo	URL
Run 27450493913	Primeiro deploy completo (merge do PR #21)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/actions/runs/27450493913
Run 27451618014	Deploy com OpenTelemetry/Jaeger (merge do PR #22)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/actions/runs/27451618014

3. Link do Vídeo

Vídeo de até 15 minutos demonstrando deploy, execução do CI/CD, consumo das APIs e escalabilidade automática, conforme o enunciado. Roteiro de gravação: [roteiro-video.md](#).

Recurso	URL
Vídeo de demonstração	<i>link será adicionado após a gravação</i>

4. Link da Documentação

Toda a documentação versionada está no próprio repositório, na pasta docs/.

4.1 Índice geral

Recurso	URL
Pasta docs/ (índice)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/tree/main/docs
Requisitos da fase 2 (enunciado transcrito)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/requisitos/fase2/desafio-tech-fase-2.md
Gap analysis — enunciado × código da fase 1 (RF-020-024, RNF-017-024, RN-018-020)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/requisitos/fase2/gap-analysis-fase-2.md
Apêndice A — funcionalidades extras da fase 2 (além do enunciado)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/entrega/fase2/apendice-funcionalidades-extras.md
Scans de segurança — fechamento da fase 2 (bateria na HEAD final)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/seguranca/scan-fase-2.md

4.2 Decisões de arquitetura da fase 2

Artefato	Decisão	URL
RFC-002	Infraestrutura e deploy da fase 2 — desenho integrado e diagrama de referência	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/rfc/fase2/rfc-002-infraestrutura-e-deploy-fase-2.md
ADR-015	Clean Architecture como arquitetura alvo (sem rewrite)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/015-arquitetura-alvo-fase-2.md
ADR-016	kind como plataforma Kubernetes (dev, vídeo e CI)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/016-plataforma-kubernetes.md
ADR-017	PostgreSQL como StatefulSet provisionado pelo Terraform	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/017-provisionamento-banco.md
ADR-018	Notificação de status por e-mail via adapter SMTP com Mailpit	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/018-notificacao-email.md
ADR-019	Pipeline de CI/CD com deploy em cluster kind efêmero no runner	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/019-pipeline-cicd-deploy.md
ADR-020	Observabilidade com OpenTelemetry e Jaeger em escopo mínimo	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/020-observabilidade-opentelemetry.md
ADR-021	Aprovação e recusa externas de orçamento via token dedicado	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/021-aprovacao-externa-orcamento.md
ADR-022	Transactional Outbox + relay para entrega de eventos de integração	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/022-transactional-outbox-relay.md
ADR-023	Rate limiter com storage compartilhado (Redis) sob HPA	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/023-rate-limiter-storage-compartilhado.md
ADR-024	Métricas de observabilidade com Prometheus e OpenTelemetry no relay	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/arquitetura/adr/fase2/024-metricas-prometheus.md

A documentação da fase 1 (Event Storming, Domain Storytelling, Linguagem Ubíqua, mapa de contextos, modelo de domínio, ADRs 001–014) permanece válida e versionada nas mesmas pastas — índice em [docs/](#).

5. Relatório de Análise de Vulnerabilidades

A postura de segurança da fase 2 é **verificada por CI, não afirmada em documento**: os seis scanners que a fase 1 rodava manualmente foram automatizados como gates de pipeline ([PR #116](#), [fecha #75](#)) e reexecutados na HEAD final, já sobre **Python 3.14** ([PR #150](#)). Os seis passaram verdes — cobrindo src/ + relay/ no SAST, dependências de runtime na SCA, a imagem 3.14 no scan de container, segredos, análise semântica e DAST contra a API viva. A sétima camada é o **SonarQube**, scan manual de fechamento de fase ([TD-010/ADR-011](#)), executado na mesma HEAD com todos os achados tratados.

5.1 Ferramentas e resultado na HEAD final

Ferramenta	Tipo	Alvo	Resultado
bandit	SAST	src/ + relay/ (10.112 LoC)	0 high / 0 medium / 0 low
pip-audit	SCA — dependências	deps de runtime resolvidas do uv.lock	0 vulnerabilidades (3 CVEs de nicegui dev-only aceitos, #112)
trivy	SCA — imagem Docker	imagem de runtime pytstop (Python 3.14)	0 HIGH/CRITICAL no gate
gitleaks	Deteção de segredos	árvore de trabalho, com allowlist	0 leaks
CodeQL	SAST semântico	python + javascript-typescript (default setup)	Analyze verde, sem alertas ativos
OWASP ZAP	DAST baseline	API viva via OpenAPI (stack compose)	0 FAIL — 2 WARN aceitos como IGNORE
SonarQube	Análise estática + security hotspots	src/ (7,4k LoC, cobertura importada)	Quality Gate Passed — 0 security, 0 reliability, coverage 95,3%; hotspots 3 → 0

No ciclo do SonarQube, a primeira análise apontou **3 security hotspots**: um na regex de extração de e-mail com backtracking polinomial (S5852) e dois em avisos de `http://` no exporter OTLP. A regex foi corrigida no código — e a mesma classe de defeito foi eliminada também na regex do scrubber de logs ([PR #155](#)); os avisos de `http://` foram revisados como seguros (tráfego gRPC intra-cluster, com endpoint externo entrando via env com https). A reanálise fechou com **0 hotspots**; o antes/depois está no Anexo B do PDF.

Gates em [security.yml](#) (pip-audit, gitleaks, trivy), [ci.yml](#) (bandit) e [full-test-ci.yml](#) (ZAP), mais o CodeQL pelo default setup do GitHub e o Dependabot mensal.

5.2 Itens de segurança endereçados

Além dos scans limpos, a auditoria de finalização ([issue #128](#)) gerou correções de segurança com teste TDD:

- **Revogação de refresh token** (CWE-613) e logout idempotente ([PR #142](#) — [#118](#)/[#121](#));
- **Corrida TOCTOU (time-of-check/time-of-use) na recusa externa de orçamento** — revalidação sob lock antes do cancelamento ([PR #142](#) — [#119](#));
- **Item de estoque desativado** barrado em OS nova e na reserva ([PR #142](#) — [#120](#));
- **Seed com denylist sensível a ambiente** — `seed_admin.py` rejeita o `ADMIN_PASSWORD` público de demo fora de `development/test` ([PR #152](#) — [#95](#); escopo por ambiente no [PR #159](#));
- **Papel de usuário fail-closed** — removido `default="admin"` do mapping, inserção sem papel passa a falhar com violação de `NOT NULL` ([PR #152](#) — [#96](#));
- **Webhook de orçamento assinado por HMAC** ([PR #114](#), TD-027), com a collection do Postman assinando via pre-request script ([PR #157](#));
- **Rate limiter global sob HPA** com storage compartilhado Redis ([PR #62](#), ADR-023);
- **Mensagens de erro sem eco de dado pessoal** — invariantes de domínio com PII usam rótulo fixo (varredura dos 75 `raise ValueError`) e o 422 de validação de schema deixou de devolver o input cru ([PR #155](#) — [#126](#));
- **Scrubber de PII nos logs ampliado** — telefones BR sem espaços ou com +55 colado passam a ser mascarados pelo valor, e os campos `telefone/celular/contato`, pelo nome ([PR #155](#) — [#99](#)).

5.3 Documentos completos

Documento	URL
Scans de fechamento da fase 2 (v2.1, HEAD final — inclui o ciclo Sonar-Qube)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/seguranca/scan-fase-2.md
Relatório de Vulnerabilidades (base-line OWASP API Top 10, fase 1)	https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2/blob/main/docs/seguranca/relatorio-vulnerabilidades.md

6. Rastreabilidade requisito → evidência

Cada requisito da fase 2 ([gap analysis](#)) está mapeado para o PR que o implementou e para a evidência principal no código; a sequência de demonstração está no [roteiro do vídeo](#).

Requisitos funcionais

ID	PR	Requisito	Evidência (arquivo / teste chave)
RF-020	#15	Abertura de OS com cliente, veículo, serviços e peças, retornando id único	CriarOrdemDTO com <code>servicos/pecas (src/ordem_servico/aplicacao/dtos.py)</code> e montagem única de itens em <code>use_cases.py</code> ; e2e tests/integracao/ <code>test_api_e2e.py::TestCriacaoOsComItens</code>
RF-021	#14	Consulta de status no vocabulário do enunciado	<code>situacao_de</code> em <code>src/ordem_servico/aplicacao/situacoes.py</code> + campo <code>situacao</code> nos 3 schemas de resposta; tests/unitarios/ <code>ordem_servico/test_presenters.py</code>
RF-022	#16	Endpoint externo de aprovação/recusa de orçamento	Rota POST <code>/api/v1/publico/ordens-de-servico/{ordem_id}/decisao-orcamento</code> em <code>src/compartilhado/interfaces/router_publico.py</code> ; use case <code>DecidirOrcamento</code> ; ADR-021
RF-023	#13	Listagem ordenada por prioridade de status, sem encerradas (exclusão lógica)	<code>_PRIORIDADE_STATUS/_ESTADOS_ENCERRADOS</code> + parâmetro <code>incluir_encerradas</code> em <code>src/ordem_servico/infraestrutura/repository.py</code> ; teste-guarda em tests/unitarios/ <code>ordem_servico/test_repository_os.py</code>
RF-024	#17 , #56	Notificação de atualização de status por e-mail	Notificação via Transactional Outbox : a <code>UnitOfWork</code> grava o <code>IntegrationEvent</code> na mesma transação da mudança de OS e o relay entrega o e-mail, com idempotência e retries (ADR-022). Handler de e-mail em <code>relay/handlers.py</code> + adapter SMTP em <code>infraestrutura/</code> ; tests/unitarios/ <code>ordem_servico/test_notificacoes.py</code>

Requisitos não funcionais

ID	PR	Requisito	Evidência (arquivo / teste chave)
RNF-017	#12	Clean Architecture formalizada e verificada	Contratos de camadas em <code>[tool.importlinter]</code> (<code>pyproject.toml</code>), verificados na CI (step <i>Architecture contracts</i> , <code>lint-imports</code> ; paridade local via <code>make lint-arch</code>); ADR-015
RNF-018	#13-#17 (transversal)	Testes dos fluxos críticos mantidos na evolução	Gate de 95% em <code>.coveragerc</code> (1.617 testes unitários + 163 de integração na HEAD final); cobertura de 95,3% em <code>src/</code> medida no fe-

ID	PR	Requisito	Evidência (arquivo / teste chave)
RNF-019	#18	Dockerfile e docker-compose revisados (healthcheck do app)	chamento (Anexo A) — CI verde na main (run 28637221227) HEALTHCHECK no Dockerfile + bloco healthcheck do serviço app no docker-compose.yml, ambos probando /api/v1/saude
RNF-020	#19	Manifests K8s: Deployment, Service, ConfigMap, Secret, HPA	k8s/ — namespace.yaml, deployment.yaml, service.yaml, configmap.yaml, secret.yaml, hpa.yaml, jobs/migration-job.yaml, mailpit.yaml, jaeger.yaml, relay.yaml, redis.yaml, prometheus.yaml
RNF-021	#20	IaC: Terraform provisiona cluster e banco, documentado	infra/ — cluster kind + namespace + Secret + StatefulSet PostgreSQL + Service num único apply; recursos documentados em infra/README.md
RNF-022	#21	CI/CD: build, testes, imagem, deploy de banco e app, manifests	.github/workflows/cd.yml + alvos make k8s-up/k8s-smoke/cd-local espelhando o workflow
RNF-023	#19	HPA-readiness: probes e resources no Deployment	Liveness/readiness em /api/v1/saude + requests/limits em k8s/deployment.yaml; metrics-server instalado pelo fluxo de deploy
RNF-024	#19 , #62	Statelessness para escala horizontal	JWT stateless com denylist no PostgreSQL (fase 1); ENCRYPTION_KEY única e estável via k8s/secret.yaml; pool dimensionado para o pior caso do HPA (DB_POOL_SIZE); rate limiter com storage compartilhado (Redis) via storage_uri → limite por IP correto e global sob HPA (não diverge entre réplicas), com degradação graciosa para per-réplica se o Redis cair (ADR-023 , TD-016)

Regras de negócio

ID	PR	Regra	Evidência (arquivo / teste chave)
RN-018	#13	Prioridade Em execução > Aguardando aprovação > Em diagnóstico > Recebida; mais antigas primeiro, desempate por id	CASE de prioridade + criado_em ASC, id em src/ordem_servico/infraestrutura/repository.py
RN-019	#13	Exclusão lógica de FINALIZADA/ENTREGUE (nenhum delete físico)	Filtro de consulta notin(_ESTADOS_ENCERRADOS); incluir_encerradas=true prova que as linhas permanecem
RN-020	#13 (ratificada no ADR-021)	Status extras: complementar ordem com Aguardando aprovação; CANCELADA excluída como encerrada	Teste-guarda de totalidade dos 8 estados em tests/unitarios/ordem_servico/test_repository_os.py

Além dos requisitos: observabilidade com OpenTelemetry. **Traces** de FastAPI e SQLAlchemy no Jaeger ([ADR-020](#), [PR #22](#)); **métricas** do relay no Prometheus — profundidade da outbox, idade do pendente mais antigo, DLQ (dead-letter queue) e contadores de entrega/falha/retry, via MeterProvider OTel + PrometheusMetricReader ([ADR-024](#), [PR #66](#), TD-022). Ambos rodam no cluster de demo e são demonstrados no bloco 6 do vídeo.

Qualidade além do escopo — dívida técnica endereçada

O backlog de dívida técnica é mantido como um ledger versionado em [docs/tech-debt/README.md](#), com itens classificados e rastreados desde a fase 1. O ledger registra hoje **28 itens resolvidos** e **6 abertos** (todos deliberados e justificados, sem impacto de produção no caminho suportado; os achados da [auditoria pré-entrega](#) foram endereçados por completo). Fora do escopo exigido pela fase 2, o grupo amortizou parte expressiva desse backlog — incluindo um débito tático de DDD herdado da fase 1 — atacando-o em tiers priorizados ([plano-ataque.md](#)) e mantendo o ledger fiel ao código. Os principais:

Item	PR	O que foi feito
TD-009 (DDD tático — fase 1)	#48	Emissão dos eventos de criação que faltavam no event storming, ClienteCadastrado e ServicoCadastrado, via factory criar() nos agregados, com payload sem PII e teste de regressão
TD-008 (Transactional Outbox — RF-018)	#56	Dispatch de eventos passou a usar outbox transacional + processo relay (claim-then-deliver, head-of-line, backoff/DLQ, idempotência via processed_events, LISTEN/NOTIFY) — elimina o dual-write das notificações
TD-016 (rate limiter sob HPA — RNF-024)	#62	Rate limiter slowapi com storage compartilhado (Redis), opt-in por env, com degradação graciosa; limite correto entre réplicas
TD-022 (métricas do relay — observabilidade)	#66	Pilar de métricas no relay: Prometheus no cluster + MeterProvider OTel/ PrometheusMetricReader expondo / metrics, opt-in por env; complementa a ADR-020 na parte de métricas (ADR-024)
TD-015 (corrida de migração multi-réplica — infra)	#64	Migração movida do entrypoint do pod para um Job dedicado (pytstop-migrate), aplicado antes do rollout com kubectl wait --for=condition=complete; RUN_MIGRATIONS_ON_STARTUP/ RUN_SEED_ON_STARTUP passam a false no cluster — resolve a corrida com N réplicas

Item	PR	O que foi feito
TD-011 (DAST automatizado — segurança)		OWASP ZAP baseline automatizado no <code>full-test-ci</code> (DAST contra a stack compose), relatório como artefato + <code>alvo make dast</code> para paridade local
TD-021 (relay HA — fencing de lease)	#66	Fencing de lease na entrega (re-lock <code>FOR UPDATE SKIP LOCKED</code> + checagem de status na transação por-linha) torna <code>replicas>1</code> seguro sem duplicar entrega, sem mudança de schema
TD-023 (rate-limit por cliente real atrás de proxy — segurança/RNF-024)	#67	<code>ProxyHeadersMiddleware</code> do <code>uicorn</code> aplicado quando <code>TRUSTED_PROXIES</code> está configurado, reescrevendo <code>request.client</code> a partir do <code>X-Forwarded-For</code> confiável; default vazio (não confia em XFF, sem spoof)
TD-005 (orçamento em coluna Text — domínio)	#68	Coluna <code>orcamento_json</code> migrada de Text para <code>jsonb</code> nativo (migração 004), removendo a camada manual <code>json.dumps/json.loads</code> no mapping; serialização passa a seguir o padrão de <code>outbox.payload</code>
TD-007 (contato como primitivo — DDD)	#70	Contato Value Object (texto livre validado, <code>__repr__</code> PII-safe) substitui o contato: <code>str</code> primitivo no agregado <code>Cliente</code> , persistindo na mesma coluna via shadow + event listeners (padrão CPF/Placa, sem migração)
TD-010 (SonarQube como gate de CI — segurança)	#71	Fechado por decisão : SonarQube/SonarCloud não vira gate de CI (repo privado + custo desproporcional ao MVP); a análise estática em CI é <code>CodeQL</code> + <code>ruff</code> + <code>bandit</code> , com SonarQube como scan manual de fechamento
Reconciliação do ledger	#47	Auditoria item a item de <code>tech-debt/README.md</code> contra o código: TD-003 (CSP) e TD-017 (PII no OpenTelemetry) marcados como resolvidos (já implementados) e TD-002/004/005/008/016 corrigidos para refletir o estado real
TD-019 (Clean Architecture)	#50	Extração de <code>PasswordHasherPort/JWTServicePort</code> na autenticação, removendo o último acoplamento aplicação → infraestrutura; o contrato forbidden do <code>import-linter</code> passou a verificá-lo globalmente em todos os contextos, reforçando a RNF-017
#75 (gates de segurança em CI)	#116	Automação dos scanners que os docs de segurança citavam mas nenhum workflow rodava: pip-audit (CVE em deps — pegou e corrigiu 5 CVEs reais em <code>cryptography/starlette</code>), gitleaks (segredos) e trivy (CVE na imagem) em <code>security.yml</code> ; escopo do bandit ampliado (<code>src ui relay scripts</code>); CodeQL confirmado no default setup do GitHub + <code>make codeql-quality</code> local aplicando as

Item	PR	O que foi feito
		supressões de falsos positivos que o default setup não permite; Dependabot habilitado (hoje com cadência mensal)

A tabela acima destaca os itens de maior valor; o conjunto completo dos **28 resolvidos** (incluindo itens fechados nas campanhas de finalização da fase 2 e pendências herdadas da fase 1) está no ledger.

Isso concretiza a Boy Scout Rule registrada na estratégia de pagamento do ledger de dívida técnica: cada evolução deixa o código e a documentação melhores do que os encontrou.

7. Desenho da arquitetura

Diagrama de referência da fase 2 — pipeline de deploy, infraestrutura provisionada e workloads no cluster. Fonte única: RFC-002 §3.

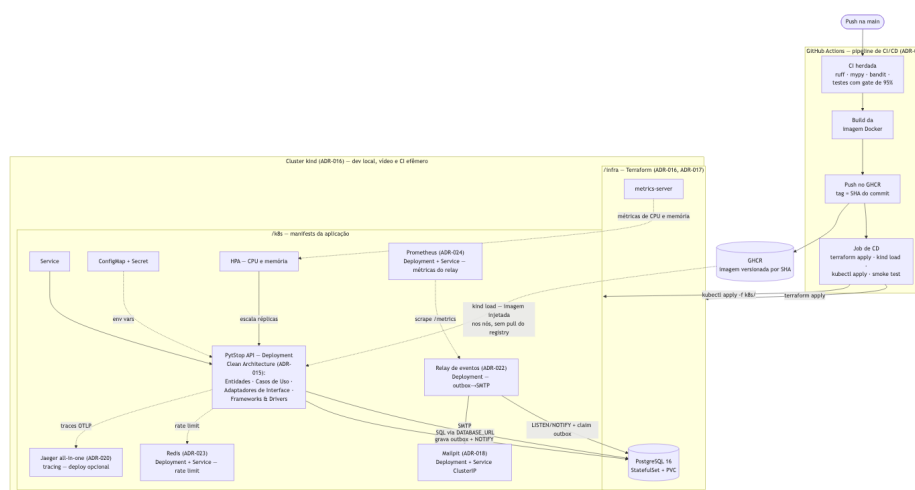


Diagrama 1 — ver fonte Mermaid no repositório

No fluxo acima, a CI atua como gate no PR (antes do merge); no push à main, CI e CD disparam em paralelo — a seta sequencial representa a ordem lógica (qualidade antes do deploy), não uma dependência entre workflows.

Evolução das camadas — da Onion (fase 1) à Clean Architecture (fase 2)

A fase 1 organizava cada contexto delimitado em quatro camadas no modelo Onion ([ADR-003](#)): dominio/, aplicacao/, infraestrutura/ e interfaces/, com a regra de dependência apontando para dentro, ports declarados em aplicacao/ e adapters concretos na borda — mas sem ordem formal entre interfaces/ e infraestrutura/.

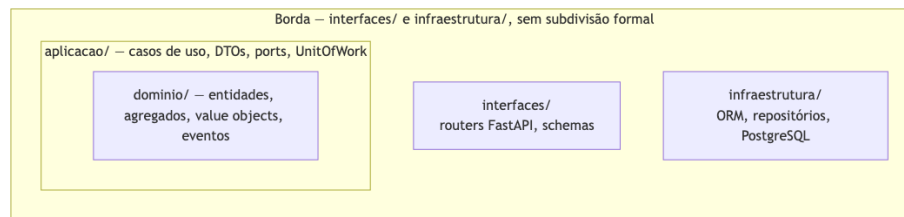


Diagrama 2 — ver fonte Mermaid no repositório

A refatoração da fase 2 ([ADR-015](#), RNF-017) adotou a Clean Architecture sem rewrite: o núcleo ports & adapters permaneceu válido, e a mudança formalizou a nomenclatura de Martin e subdividiu a borda — interfaces/ virou a camada de **Adaptadores de Interface** (controllers e presenters) e infraestrutura/ a de **Frameworks & Drivers** (gateways SQLAlchemy, ORM, conexão com o banco), cada uma com papel e regras próprios.

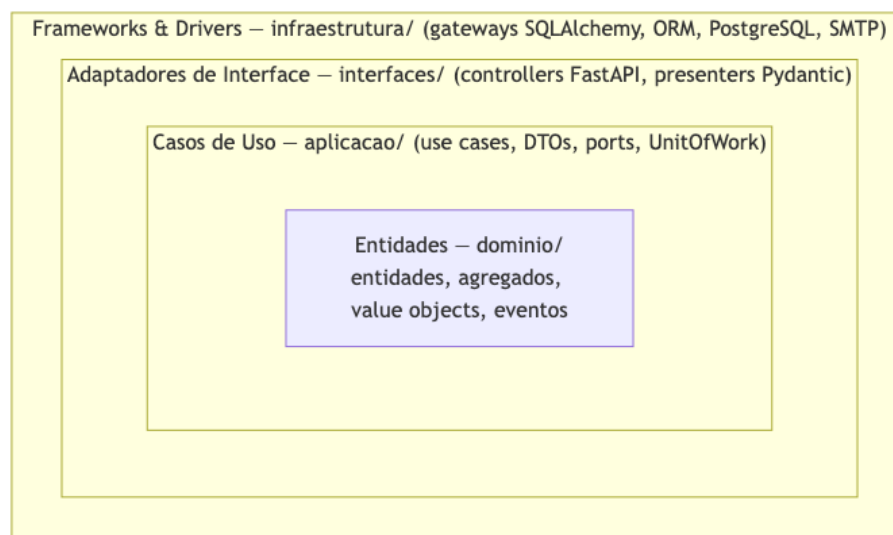


Diagrama 3 — ver fonte Mermaid no repositório

A regra de dependência deixou de ser convenção e virou gate: o import-linter roda na CI com três contratos — camadas interfaces → aplicacao → dominio em todos os contextos (inclusive o shared kernel), proibição de dominio/ e aplicacao/ importarem infraestrutura/, e independência entre contextos delimitados ([ADR-015](#)).

8. Conteúdo do PDF de submissão

O PDF entregue no portal do aluno é gerado a partir deste documento pelo `scripts/build-entrega-pdf.sh`, que acrescenta uma **capa ABNT** no início, renderiza os diagramas Mermaid como imagens, converte os links relativos em absolutos e anexa os apêndices de evidência. A seção 9 (Pendências) é um checklist interno da equipe e **não** é incluída no PDF submetido.

O PDF contém os três itens exigidos pelo enunciado:

1. **Link do repositório GitHub** compartilhado com o usuário `soat-architecture`: <https://github.com/fiap-postech-sw-architecture/postech-sw-arch-p2>
2. **Desenho da arquitetura** com os recursos escolhidos (seção 7 — kind, Terraform, GHCR, manifests K8s com HPA, Mailpit, Jaeger, Prometheus).
3. **Link do vídeo** de até 15 minutos apresentando a solução (seção 3 — preenchido após a gravação).

Mais três anexos de evidência de profundidade:

- **Anexo A — Scans de Segurança da Fase 2**: bateria de fechamento na HEAD final ([scan-fase-2.md](#)).
- **Anexo B — Evidências Visuais**: capturas da demonstração no cluster (pipeline verde, HPA escalando 1 → 5, traces no Jaeger, e-mails no Mailpit, métricas no Prometheus e o antes/depois do SonarQube — hotspots 3 → 0, Quality Gate Passed).
- **Anexo C — Funcionalidades Extras da Fase 2**: catálogo além do enunciado ([apendice-funcionalidades-extras.md](#)).

Anexo A — Scans de Segurança da Fase 2

Versão: 2.1 — adiciona a **sétima camada**: SonarQube como scan manual de fechamento (TD-010/[ADR-011](#)) executado em 02/07/2026, com Quality Gate **Passed** e os 3 security hotspots levados a zero (1 corrigido no código, 2 revisados como seguros com justificativa) — antes/depois no Anexo B do documento de entrega.

Versão 2.0 — bateria de fechamento **reexecutada na HEAD final da fase 2** (commit [5404826](#), 02/07/2026), já sobre **Python 3.14** ([PR #150](#)). Sucede a versão 1.0 (12/06/2026), que cobria só `src/+ui/` e não reexecutou trivy; esta versão fecha o escopo — `src/+relay/` no SAST, deps de runtime na SCA, imagem 3.14 no scan de container, segredos, CodeQL e DAST — e incorpora os PRs de segurança posteriores a 12/06. Complementa o [relatório de vulnerabilidades](#) da fase 1, que permanece válido para o baseline OWASP API Top 10 do MVP.

Escopo

Reexecução completa dos scans automatizados de segurança sobre o código da fase 2 na HEAD final, cobrindo as seis camadas do pipeline ([ADR-011](#)):

- **SAST** (bandit) sobre `src/ + relay/` — o `relay/` (Transactional Outbox, [PR #56/ADR-022](#)), ausente da bateria de 12/06, agora é varrido de primeira;
- **SCA de dependências** (pip-audit) sobre as dependências de **runtime** (produção), pós-migração consolidada de deps ([#143](#), redis 8) e upgrade do NiceGUI 3 ([#149](#));
- **SCA de imagem** (trivy) sobre a imagem de runtime **agora em Python 3.14** ([#150](#));
- **Deteção de segredos** (gitleaks) sobre a árvore de trabalho;
- **SAST semântico** (CodeQL, python + javascript-typescript);
- **DAST** (OWASP ZAP baseline) contra o OpenAPI vivo da stack compose.

Diferentemente da bateria de 12/06 — um scan local pontual — a bateria de fechamento **é o próprio CI**: a partir de [#116](#) (fecha [#75](#)) os gates de pip-audit, gitleaks e trivy rodam em `security.yml`; o bandit em `ci.yml` (job `security`, escopo `src/ ui/ relay/ scripts/`); o CodeQL pelo *default setup* do GitHub (jobs `Analyze (python)` e `Analyze (javascript-typescript)`); e

o ZAP baseline em `full-test-ci.yml` (TD-011, PR #65). Todos os seis passaram **verdes na HEAD final** — a evidência abaixo é o resultado dessas execuções, e não mais um scan manual sem trilha.

Nota sobre os relatórios versionados: os JSON em `docs/seguranca/*-report.json` (trivy, pip-audit, ZAP, gitleaks, bandit) são os artefatos da fase 1, preservados como baseline histórico — não são reescritos a cada bateria (o CI publica os seus próprios como artefatos de run). Os números desta versão 2.0 refletem o **estado da HEAD final** (lockfile e imagem em 3.14), que diverge do snapshot da fase 1 nos JSON versionados.

Resumo

Ferramenta	Tipo	Alvo	Resultado na HEAD final
bandit 1.9.4	SAST	src/ + relay/ (10.112 LoC)	0 high / 0 medium / 0 low — nenhum achado
pip-audit	SCA (deps)	dependências de runtime resolvidas do <code>uv.lock</code>	0 vulnerabilidades (3 CVEs de nicegui dev-only aceitos, #112)
trivy	SCA (imagem)	imagem de runtime <code>pytstop</code> (Python 3.14)	0 HIGH/CRITICAL no gate (ignore-unfixed + <code>.trivyignore</code>)
gitleaks	Segredos	árvore de trabalho (<code>gitleaks dir</code>) com <code>allowlist</code>	0 leaks
CodeQL	SAST semântico	python + javascript-typescript (default setup)	Analyze verde — sem alertas de segurança ativos
OWASP ZAP	DAST baseline	API viva via OpenAPI (stack compose)	0 FAIL — 2 WARN aceitos como IGNORE (<code>.zap/rules.tsv</code>)
SonarQube (Community, local)	Análise estática + hotspots	src/ (7,4k LoC, cobertura importada)	Quality Gate Passed — 0 security, 0 reliability, coverage 95,3%; hotspots 3 → 0 (1 FIXED, 2 SAFE)

Referência da última execução verde: `check-runs` do commit `5404826` — security, pip-audit (CVE em dependências), gitleaks (segredos), trivy (CVE na imagem), Analyze (python), Analyze (javascript-typescript) e o `full-test-ci` (que hospeda o DAST) todos success.

Análise Estática (bandit)

Gate do projeto (make security, espelhado no CI): `bandit -r src/ ui/ relay/ scripts/ ... -c pyproject.toml --severity-level high` — **nenhum achado high**. A varredura completa de `src/ + relay/` sem filtro de severidade (10.112 LoC) fecha em **0 high / 0 medium / 0 low**: nenhum resultado. O `relay/`, introduzido depois da bateria de 12/06, entra limpo.

A UI (`ui/`, dev-only, fora do runtime de produção e não empacotada pelo `pyproject.toml`) mantém o único achado **low** já aceito na versão 1.0:

Arquivo	ID	Severidade	Análise
ui/config.py:52	B105 (hardcoded password)	Low	Fallback dev-only do <code>storage_secret</code> da UI NiceGUI, usado somente quando <code>UI_STORAGE_SECRET</code> não está definido. Valor auto-documentado (<code>pytstop-ui-dev-only-...</code>). Aceito.

Reprodução:

```
make security
```

Auditoria de Dependências (pip-audit)

O gate de CI (`security.yml`) audita apenas as dependências de **runtime** — `uv export --no-dev --no-emit-project` gera `requirements-prod.txt`, que exclui o tooling de teste e a UI NiceGUI (dev-only) — porque essa é a superfície de ataque de produção:

```
uv export --frozen --no-emit-project --no-dev --no-hashes --format requirements-txt -o requirements-prod.txt
uvx pip-audit -r requirements-prod.txt --strict
```

Resultado na HEAD final: No known vulnerabilities found. Os únicos advisories excluídos são os **3 HIGH de nicegui** ([CVE-2025-66645](#), [CVE-2026-21873](#), [CVE-2026-25732](#)) — a UI é dev-only e não consta no lock-file de produção; ficam como `--ignore-vuln` explícito (cinto-e-suspensório, [#112](#)).

Este resultado limpo é o **estado consolidado** de duas ondas de upgrade posteriores à v1.0:

Onda	PR	O que mudou
Gate de CI + 5	#116	A automação do pip-audit no CI (#75) pegou 5 CVEs reais e forçou o upgrade — cryptography 46→49, starlette 1.0→1.3.1, fastapi →0.138.2 — além dos bumps de <code>pyjwt/urllib3/idna/mako</code> já feitos na v1.0

Onda	PR	O que mudou
CVEs reais		
Migração consolidada de deps	#143	Leva do Dependabot consolidada (redis 8, entre outros), com Dependabot mensal ativo
NiceGUI 3	#149	Upgrade nicegui 2.24 → 3.14 — removeu a transitiva vbuild (que usava pkgutil.find_loader, extinto no Python 3.14), destravando a migração 3.14
Pisos de deps + Terraform action	#151	Alinha pisos de pdwlib/testcontainers; setup-terraform@v4

Versões-chave resolvidas no uv.lock da HEAD final: pyjwt 2.13.0, starlette 1.3.1, cryptography 49.0.0, fastapi 0.139.0, urllib3 2.7.0, idna 3.18, mako 1.3.12, redis 8.0.1. A suíte completa (1.617 testes unitários + 163 de integração, contagem via pytest --collect-only) segue verde após os bumps, validando ausência de regressão.

Scan de Imagem (trivy)

Reexecutado nesta bateria (a v1.0 havia pulado o trivy): a imagem de runtime agora é **Python 3.14** ([#150](#) — builder ghcr.io/astral-sh/uv:python3.14-bookworm-slim + runtime python:3.14-slim). O gate ([security.yml](#)) constrói a imagem e reprova em HIGH/CRITICAL:

```
docker build -t pytstop:ci-scan .
trivy image --severity HIGH,CRITICAL --ignore-unfixed --
ignorefile .trivyignore --exit-code 1 pytstop:ci-scan
```

Resultado na HEAD final: 0 HIGH/CRITICAL no gate. Os CVEs de pacotes de SO da imagem base sem fix upstream (ncurses, systemd) são descartados por ignore-unfixed e listados explicitamente em [.trivyignore](#) (CVE-2025-69720, CVE-2026-29111) — não usados pelo runtime FastAPI/uvicorn (entrypoint é uvicorn direto, sem systemd), aceite herdado de [#113](#).

Detecção de Segredos (gitleaks)

O gate ([security.yml](#)) varre a **árvore atual** (não o histórico) com a config do repo — barra segredos novos entrando por PR sem re-flagar dev-secrets antigos já cobertos pela allowlist:

```
gitleaks dir . --config .gitleaks.toml --redact --no-banner --
verbose
```

Resultado na HEAD final: 0 leaks. A `.gitleaks.toml` mantém a allowlist documentada dos falsos positivos e valores de demo deliberados da fase 1 (digests do relatório trivy, estado local do Terraform gitignored, token de demo do webhook de orçamento — todos auto-documentados como `...-nao-usar-em-producao`).

SAST Semântico (CodeQL)

O CodeQL roda pelo **default setup** do GitHub code scanning (decisão de [#116/TD-010](#): sem workflow `codeql.yml` avançado, que conflita com o default setup). Os jobs `Analyze (python)` e `Analyze (javascript-typescript)` estão **verdes na HEAD final**, sem alertas de segurança ativos. Paridade local via `make codeql-quality`, que aplica as supressões de falso positivo que o default setup não permite.

A API REST de code scanning responde 403 "Advanced Security must be enabled" para este repositório privado — mensagem enganosa: o default setup **está ativo** e os jobs `Analyze` rodam e reprovam normalmente no CI (é a razão de não se criar um `codeql.yml` avançado). A evidência do resultado são os check-runs verdes, não a API de alerts.

DAST (OWASP ZAP baseline)

O ZAP baseline (scan passivo) roda continuamente no `full-test-ci` contra o OpenAPI vivo da stack compose ([TD-011](#), [PR #65](#)), com gate por `.zap/rules.tsv`: sem `-I`, qualquer alerta que não esteja como `IGNORE` reprova a build. As duas únicas regras em `IGNORE` são os 2 `WARN` aceitos e justificados na fase 1 (falsos positivos esperados para API REST):

Plu- gin	Regra	Tratamento
10049	Non-Storable Content	IGNORE — correto para uma API com <code>Cache-Control: no-store</code>
90004	Cross-Origin-Resource-Policy Header	IGNORE — superfície interna do cluster de demo

Resultado na HEAD final: 0 FAIL — nenhum achado novo além dos 2 `WARN` aceitos; o `full-test-ci` fechou verde no commit `5404826`. Baseline de referência da fase 1: 65 PASS / 0 FAIL / 2 `WARN` (49 URLs via OpenAPI). Paridade local via `make dast`.

SonarQube (scan manual de fechamento)

O SonarQube **não é gate de CI** por decisão registrada (TD-010, [ADR-011](#)): repo privado torna o SonarCloud pago e um servidor self-hosted é desproporcional ao MVP. Ele roda como **scan manual de fechamento de fase** — SonarQube Community local (docker) + sonar-scanner com o [sonar-project.properties](#) versionado e o coverage.xml da suíte unitária importado.

Execução de fechamento da fase 2 (02/07/2026) — Quality Gate Passed: Security **0** (rating A), Reliability **0** (A), Maintainability 147 code smells (A, informativo), Coverage **95,3%**, Duplications **0,0%**. A primeira análise apontou **3 security hotspots** (“to review” — pontos de atenção, não vulnerabilidades confirmadas), todos tratados no fluxo de revisão da própria ferramenta:

Hotspot	Regra	Tratamento
Regex de extração de e-mail com backtracking polinomial (notificacoes.py)	S5852 (DoS)	Corrigido no código: domínio casado label a label (. fora da classe de caracteres) elimina o backtracking; input já limitado a 255 chars pelo VO Contato. Teste adversarial em TestExtrairEmail. Revisado como FIXED
http://jaeger:4317 como endpoint OTLP default (observability.py, 2 ocorrências)	S5332 (encrypt-data)	Revisado como SAFE: tráfego OTLP gRPC intra-cluster (o DNS jaeger só resolve dentro do cluster/compose); um collector externo entra via OTEL_EXPORTER_OTLP_ENDPOINT com https, que desliga o modo insecure automaticamente

Resultado final: 0 hotspots a revisar, Quality Gate mantido **Passed**. O universo de cobertura foi alinhado ao gate do .coveragerc (sonar.coverage.exclusions=**/__init__.py — arquivos omitidos do gate não entram no denominador): a primeira leitura reportava 93,6% por medir um conjunto maior de arquivos do que o gate de 95% mede; alinhado, a cobertura real é **95,3%** (o gate de CI mede 97,5% incluindo ui/). O antes/depois está nas evidências visuais do Anexo B do documento de entrega ([b6b-sonarqube-quality-gate.png](#) → [b6b-sonarqube-hotspots-zerados.png](#)).

Itens de Segurança Endereçados na Fase 2

Além dos scans limpos, a fase 2 fechou um conjunto de correções de segurança/correção — os bugs confirmados da [auditoria de finalização](#) (issue [#128](#)) viraram issues no GitHub e foram corrigidos com teste TDD (red→green):

Item	PR (issue)	Correção
Revogação de refresh token (CWE-613)	#142 (#118 , #121)	Logout passa a revogar o refresh (best-effort, escopado ao dono) e vira idempo-

Item	PR (issue)	Correção
		tente (guard antes do INSERT — logout duplo não estoura o UNIQUE do JTI)
TOCTOU na recusa externa de orçamento	#142 (#119)	A recusa externa revalida o estado de espera sob lock antes de delegar o cancelamento — fecha a janela que cancelava OS já em execução
Item de estoque desativado em OS	#142 (#120)	Peça desativada não entra em OS nova nem é reservada (ItemEstoqueDTO.ativo + rejeição em _montar_item + defesa em ItemEstoque.reservar)
Instância stale sob FOR UPDATE	#142 (#117)	populate_existing=True no ramo com_lock + listeners de refresh no mapping — sem isso o SELECT FOR UPDATE devolvia a instância stale do identity map, derrotando a defesa de concorrência
Seed com senha de demo pública	#152 (#95)	seed_admin.py rejeita o ADMIN_PASSWORD de demo commitado (pytstop-admin-demo-2026); teste-guarda falha se removerem o valor da denylist
Papel de usuário fail-closed	#152 (#96)	Removido default="admin" da coluna papel no mapping de auth — inserção sem papel vira NOT NULL (fail-closed) em vez de admin silencioso, completando o fail-safe do #84
Webhook de orçamento assinado	#114	Aprovação/recusa externa passa a exigir assinatura HMAC (X-Webhook-Signature + X-Webhook-Timestamp) — ADR-021 , TD-027
Rate limiter global sob HPA	#62 , #67	Storage compartilhado (Redis) via storage_uri — limite por IP correto e global entre réplicas, com degradação graciosa se o Redis cair (ADR-023); IP real atrás de proxy via ProxyHeadersMiddleware quando TRUSTED_PROXIES está setado

Complementam os controles já entregues antes desta bateria:
 ENCRYPTION_KEY obrigatória em produção + decrypt sem fail-open ([#103](#)),
 erasure LGPD admin-only com trilha de auditoria ([#104](#)), pré-hash bcrypt de
 72 bytes + refresh não vale como access ([#105](#)), securityContext nos work-
 loads k8s ([#108](#)) e scrub de PII em tracebacks/logs ([#86](#)).

Relação com Outros Documentos

- [Relatório de vulnerabilidades \(fase 1\)](#) — baseline OWASP API Top 10, trivy, ZAP e SonarQube
- [Plano de segurança](#) — modelo de ameaças e controles
- [ADR-011](#) — pipeline de segurança em camadas

- Dívida técnica — débitos de segurança aceitos e rastreados

Anexo B — Evidências Visuais

Capturas da demonstração no cluster kind (make cd-local), na mesma sequência do roteiro do vídeo. Fontes versionadas em docs/entrega/fase2/evidencias/.

B1 — Pipeline verde na main

```
gh run list -- workflows na main (CI/CD/Security/CodeQL/full-test)

$ gh run list --branch main --limit 6
completed success docs(entrega): fechamento da fase 2 - segurança, extras, capa+anexos - Security main push 28620840918 1m14s
completed success docs(entrega): fechamento da fase 2 - segurança, extras, capa+anexos - CI main push 28620840911 2m25s 28620840887
completed success docs(entrega): fechamento da fase 2 - segurança, extras, capa+anexos - CD main push 28620840887 3m19s 28620840839
completed success Code Quality: Push on main CodeQL main dynamic 28620840436 1a50s 2025-07-02T20:52:41Z
completed success docs(entrega): fechamento da fase 2 - segurança, extras, capa+anexos - full-test (ci) main push 28620840839 4m44s
completed success chore: housekeeping da entrega - README 3.14, seed denyList, papel s... Security main push 28617375645 56s
```

CI, CD, Security, CodeQL e full-test verdes na main

B2 — HPA escalando 1→5 sob carga

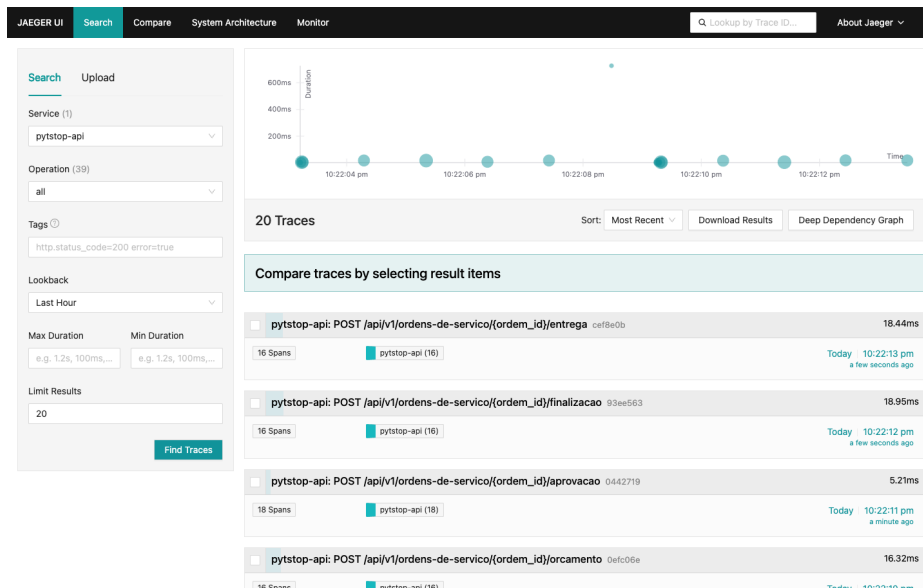
```
kubectl -- HPA pytstop-api escalando sob carga (kind)

$ kubectl -n pytstop get hpa
NAME      REFERENCE          TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
pytstop-api  Deployment/pytstop-api  cpu: 43%/70%, memory: 32%/80%    1         5         5         6d15h

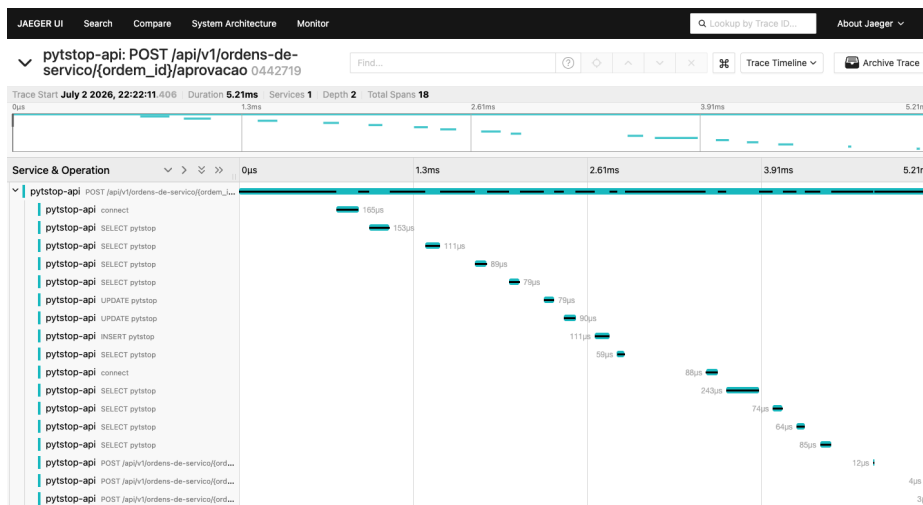
$ kubectl -n pytstop get pods -l app=pytstop-api
NAME                                READY  STATUS   RESTARTS  AGE
pytstop-api-64bbdff5b-25glz         1/1    Running  1 (2m4s ago)  5m57s
pytstop-api-64bbdff5b-d8dwh         1/1    Running  1 (2m4s ago)  5m43s
pytstop-api-64bbdff5b-mhbwet        1/1    Running  4 (2m2s ago)  2d21h
pytstop-api-64bbdff5b-mzpmk         1/1    Running  1 (2m2s ago)  5m43s
pytstop-api-64bbdff5b-qnpqd         1/1    Running  1 (2m3s ago)  5m57s
```

kubecttl: HPA pytstop-api com 5 replicas sob carga

B3 – Traces no Jaeger

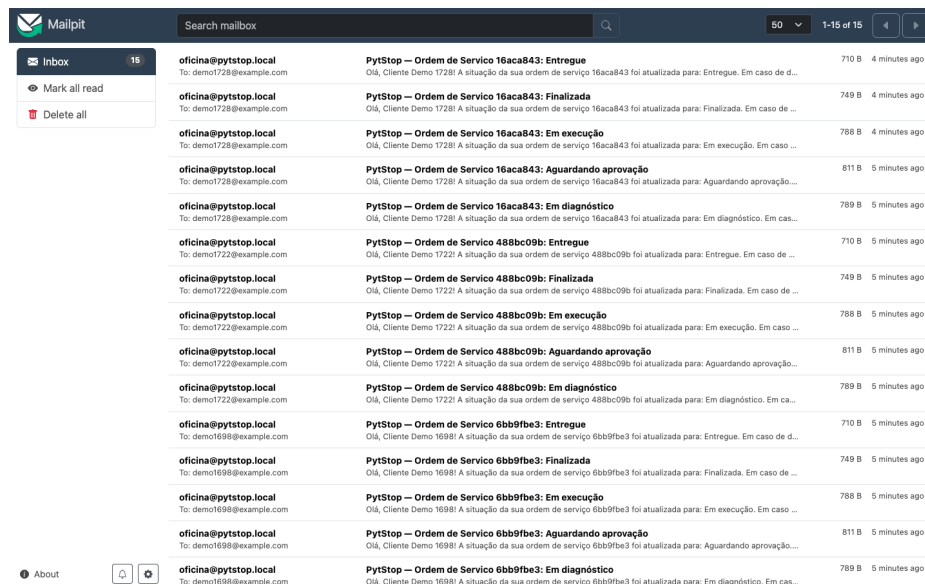


Busca no Jaeger: traces das transicoes de OS



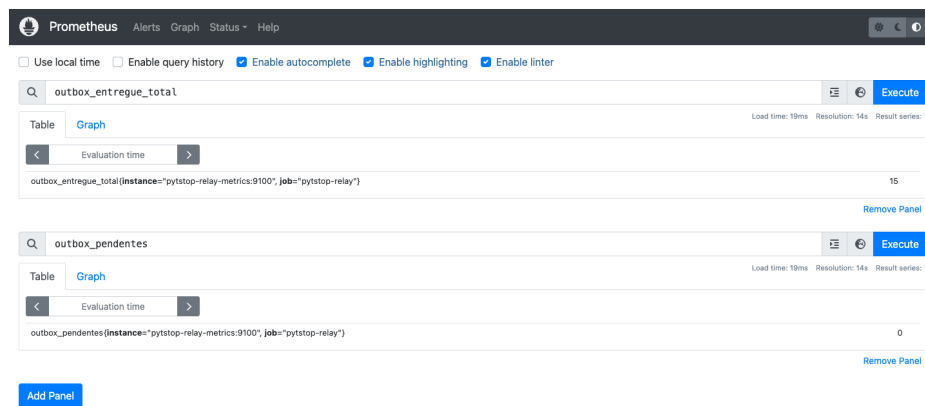
Trace da aprovacao com 18 spans fastapi+sqlalchemy

B4 — E-mails no Mailpit (um por transição de OS)



Mailpit com 15 e-mails de transicao

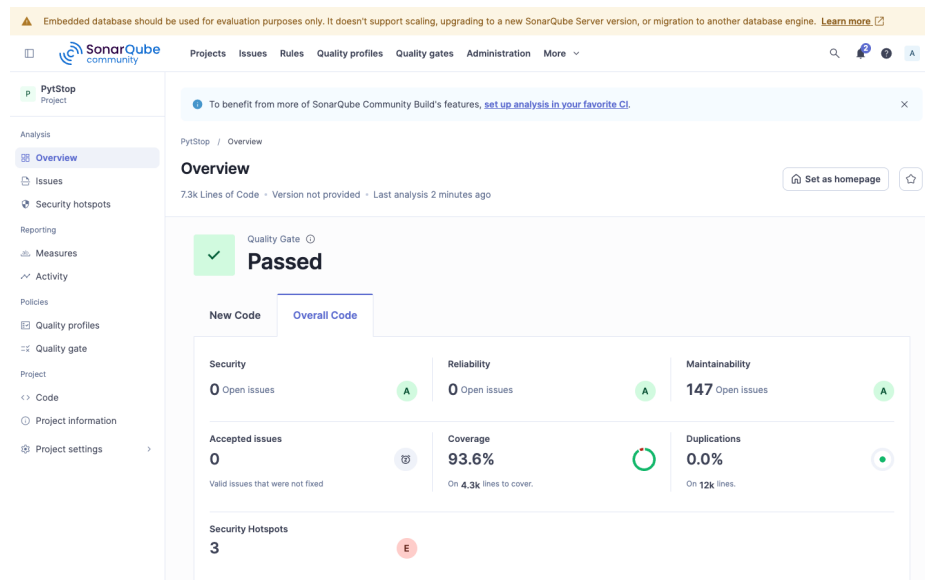
B5 — Métricas do outbox no Prometheus



outbox_entregue_total=15 e outbox_pendentes=0

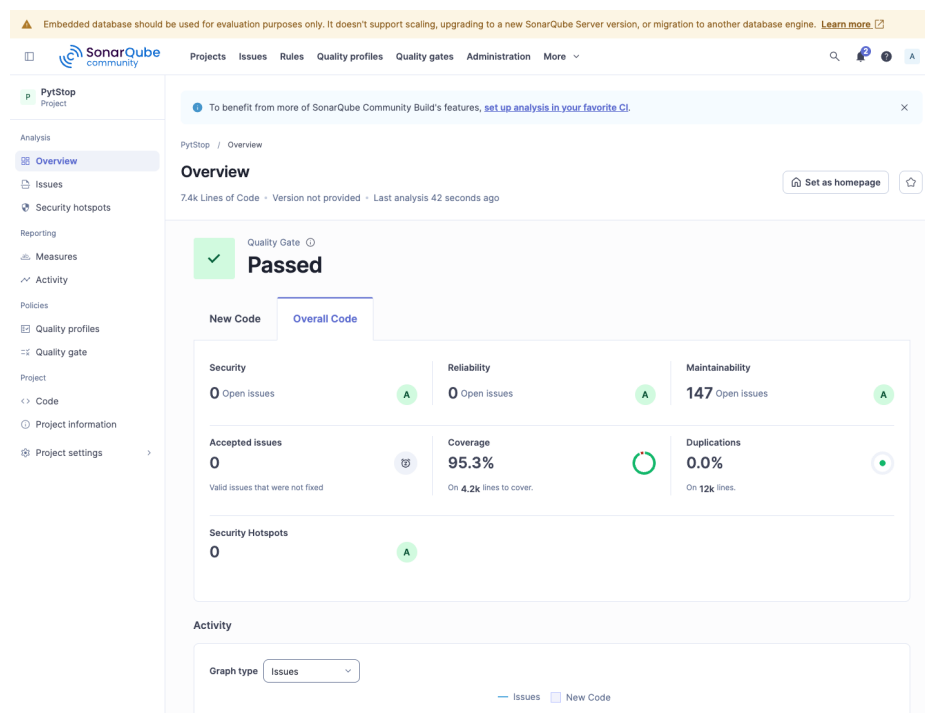
B6 — SonarQube: scan de fechamento, antes e depois

Antes — primeira análise da HEAD final: Quality Gate Passed com **3 security hotspots** a revisar.



Antes: Quality Gate Passed, 3 hotspots a revisar

Depois — hotspots tratados no fluxo da ferramenta (1 corrigido no código — regex de e-mail sem backtracking polinomial, S5852; 2 revisados como seguros — OTLP intra-cluster), universo de cobertura alinhado ao gate e reanálise: **0 hotspots**, gate mantido Passed, coverage 95,3%. Detalhes na seção SonarQube do Anexo A.



Depois: 0 hotspots a revisar, Quality Gate Passed

Anexo C — Funcionalidades Extras da Fase 2

Documento complementar ao Documento de Entrega — Fase 2. Cataloga as funcionalidades implementadas **além** dos entregáveis obrigatórios do enunciado da Fase 2, cada uma ancorada em evidência verificável (ADR, PR, commit ou arquivo). É o análogo, para a Fase 2, do Apêndice da Fase 1.

O que é este apêndice

O enunciado da Fase 2 ([desafio-tech-fase-2.md](#)) exige um conjunto fechado de itens: refatoração para Clean Architecture, cinco APIs de OS, containerização (Dockerfile + docker-compose), manifests Kubernetes (Deployment, Service, ConfigMap, Secret, HPA), Terraform para cluster e banco, pipeline de CI/CD e README com desenho da arquitetura, instruções e link do vídeo. A rastreabilidade desses itens obrigatórios está na seção 6 do documento de entrega (RF-020–024, RNF-017–024, RN-018–020).

Este apêndice reúne o que o grupo entregou **fora** desse escopo — decisões de robustez, observabilidade, segurança e infraestrutura que não eram cobradas mas evidenciam profundidade de engenharia. O critério de “extra” é objetivo: qualquer item que o enunciado não pede explicitamente. Boa parte nasceu do ledger de dívida técnica versionado ([docs/tech-debt/README.md](#), 26 itens resolvidos) e da auditoria pré-entrega ([auditoria-pre-entrega-fase2.md](#)), atacados em tiers priorizados após a base obrigatória já estar verde. Nenhum era exigido pela fase.

Critério e organização

Cada extra abaixo traz **motivação** (por que foi feito, já que não era obrigatório) e **evidência** (ADR/PR/commit/arquivo que comprova a implementação no repositório). Os extras estão agrupados por tema. A numeração de PR refere-se aos PRs do repositório da Fase 2 na org `fiap-postech-sw-architecture`.

A.1 Entrega durável de eventos e resiliência (4 extras)

1. Transactional Outbox + processo relay

- **Motivação:** o enunciado pede notificação de status “via alguma ferramenta como email” (RF-024). A entrega ingênua — enviar o e-mail dentro da transação de negócio — cria dual-write: a transição da OS pode commitar e o e-mail falhar (ou vice-versa). O grupo adotou o padrão Transactional Outbox para garantir entrega *at-least-once* sem acoplar o envio à transação.
- **Evidência:** [ADR-022](#). A UnitOfWork grava o IntegrationEvent na tabela outbox na mesma transação da OS (migração `003_outbox.py`); o relay (python `-m relay`) faz *claim-then-deliver* com FOR UPDATE SKIP LOCKED, head-of-line, backoff/DLQ e idempotência via processed_events, com notificação proativa por LISTEN/NOTIFY. Código em [relay/processador.py](#) e [relay/listener.py](#). O dispatcher síncrono foi congelado vazio (EventDispatcher(handlers=())), eliminando o dual-write. TD-008 (PR #56, commit 30c94ec).

2. Fencing de lease no relay (segurança para réplicas>1)

- **Motivação:** com o relay escalado para mais de uma réplica, duas réplicas poderiam reivindicar a mesma linha da outbox e entregar o e-mail em duplicidade. O fencing torna a escala horizontal do relay segura.
- **Evidência:** TD-021, [ADR-022](#). Re-lock FOR UPDATE SKIP LOCKED + checagem de status pendente na transação por-linha (bloquear_para_entrega em [relay/processador.py](#)), serializando réplicas concorrentes sem duplicar entrega e sem mudança de schema. PR #66 (commit 3ab383d).

3. Resiliência do ciclo do relay (blip de DB não reinicia o pod)

- **Motivação:** um erro transitório de banco no meio do loop de drenagem não deveria derrubar o pod nem perder eventos — as garantias *at-least-once* + lease já cobrem a retentativa.
- **Evidência:** o while de executar_relay embrulha heartbeat + drenagem num try/except Exception que loga outbox_ciclo_falhou e faz continue; o drain inicial pré-loop fica fora (erro no boot deve falhar alto), e except Exception (não BaseException) deixa KeyboardInterrupt/SystemExit propagarem. Código em [relay/listener.py](#). Hardening operacional do TD-008 (2026-06-25).

4. Correção de swallow de exceção no envio de e-mail (retry/DLQ alcançáveis)

- **Motivação:** o handler de notificação engolia toda exceção do envio (contrato da fase síncrona antiga), o que fazia o relay marcar o e-mail como entregue mesmo com o SMTP fora — tornando retry/backoff/DLQ inalcançáveis. Bug real corrigido durante o hardening.
- **Evidência:** `NotificarMudancaDeStatus.__call__` (`src/ordem_servico/aplicacao/notificacoes.py`) passou a capturar só transporte (`except (OSError, smtplib.SMTPException)`), logar e re-raise; os early-returns de skip seguem não-fatais. Fechado por teste de integração com testcontainers (`tests/integracao/relay/test_relay_smtp_falha.py`) que prova que a falha de SMTP leva a linha a dead, não entregue.

A.2 Observabilidade (3 extras)

5. Tracing distribuído com OpenTelemetry + Jaeger

- **Motivação:** o enunciado não pede observabilidade. Com o sistema indo para Kubernetes sob HPA, traces de request e de banco são essenciais para diagnosticar latência entre réplicas.
- **Evidência:** `ADR-020`. `configurar_otel(app, engine)` em `src/compartilhado/infraestrutura/observability.py` instrumenta FastAPI + SQLAlchemy, exportando via OTLP/gRPC; default OFF por `OTEL_ENABLED` (desligado = zero import de otel). Jaeger all-in-one como workload de demo em `k8s/jaeger.yaml`. PR #22.

6. Métricas do relay em Prometheus (via OTel MeterProvider)

- **Motivação:** a outbox é um ponto operacional crítico — sem métricas de profundidade e idade do pendente mais antigo, uma DLQ crescente passa despercebida.
- **Evidência:** `TD-022`, `ADR-024`. `MeterProvider` + `PrometheusMetricReader` em `relay/metrics.py` servem `/metrics` (porta 9100): gauges de profundidade (pendentes/idade/dead) + contadores entregue/falha/dead/retry, scrapeados por `k8s/prometheus.yaml`. Opt-in por env. PR #66 (commit 3ab383d).

7. Redação de PII nos traces (query string com CPF/placa)

- **Motivação:** sem tratamento, os spans capturam a query string — que pode conter CPF/placa —, vazando PII para o backend de tracing. Defesa em profundidade LGPD.

- **Evidência:** TD-017. `_redigir_pii_da_span` (`server_request_hook` do `FastAPIInstrumentor`) redige `url.query` e remove a query de `http.target/url.path` antes do export (`src/compartilhado/infraestrutura/observability.py`). Fechado na issue #34.

A.3 Segurança de infraestrutura e supply chain (5 extras)

8. securityContext endurecido nos workloads próprios

- **Motivação:** o enunciado pede os manifests, mas não hardening. Rodar como root, com filesystem gravável e capabilities amplas é risco desnecessário no cluster.
- **Evidência:** TD-024. Pod + container securityContext (`runAsNonRoot`, `runAsUser/runAsGroup/fsGroup 1001`, `seccompProfile: RuntimeDefault`, `allowPrivilegeEscalation: false`, `readOnlyRootFilesystem`, `capabilities.drop: [ALL]`) + `emptyDir` em /tmp nos 3 workloads próprios (`api/relay/migrate`); UID/GID 1001 pinados no `Dockerfile:52`. Manifests em `k8s/deployment.yaml` e `k8s/relay.yaml:35`. Teste de mesa no kind confirmou pods Running sob o hardening. PR #108 (commit 04c2a42).

9. Webhook de decisão de orçamento assinado por HMAC

- **Motivação:** o endpoint externo de aprovação/recusa (RF-022) inicialmente usava um token estático no corpo. Sem assinatura, um payload adulterado seria aceito. A assinatura HMAC fecha adulteração e limita replay.
- **Evidência:** TD-027, [ADR-021](#) (emendada). `src/compartilhado/infraestrutura/webhook_signature.py` — HMAC-SHA256 de `{ordem_id}. {timestamp}. + body`, chave `ORCAMENTO_WEBHOOK_TOKEN` (não trafega), janela ± 5 min via headers `X-Webhook-Signature/X-Webhook-Timestamp`. PR #114 (commit 30fe5fe).

10. Gates reais de segurança no CI (pip-audit, gitleaks, trivy)

- **Motivação:** os docs de segurança citavam scanners que nenhum workflow rodava. Automatizá-los transforma intenção em gate — e já capturou CVEs reais.
- **Evidência:** `.github/workflows/security.yml` roda **pip-audit** (pegou e corrigiu 5 CVEs reais em `cryptography/starlette/fastapi`), **gitleaks** (segredos) e **trivy** (CVE na imagem); escopo do **bandit** ampliado para `src ui relay scripts`; **CodeQL** confirmado no default setup + `make codeql-`

quality local aplicando supressões de FP; **Dependabot** configurado. PR #116 (commit 931d6aa).

11. DAST automatizado (OWASP ZAP baseline) no CI

- **Motivação:** análise estática não cobre vulnerabilidades de runtime. O baseline dinâmico contra a stack real complementa bandit/CodeQL.
- **Evidência:** TD-011, [ADR-011](#). ZAP baseline no `full-test-ci` (DAST contra a stack compose), relatório como artefato + alvo `make dast` para paridade local; regras dos WARNs aceitos em `.zap/rules.tsv`. PR #65 (commit 7b0b686).

12. SBOM automatizado (CycloneDX) no CI

- **Motivação:** rastreabilidade de supply chain — saber exatamente quais dependências (e versões) entram na imagem, com política de licenças permissivas.
- **Evidência:** TD-012, [ADR-012](#). Job sbom no `.github/workflows/ci.yml` gera o SBOM CycloneDX a partir do lockfile a cada run e publica como artefato; alvo `make sbom` para geração local.

A.4 Hardening de autenticação (4 extras)

13. Papel obrigatório no registro + coluna sem default (fail-closed)

- **Motivação:** `POST /autenticacao/registrar` criava **sempre** ADMIN (default `papel=Paper.ADMIN` e ausência do campo no request), anulando o RBAC. Correção de design fail-safe.
- **Evidência:** `papel` passou a ser obrigatório em `RegistrarRequest/RegistrarDT0`; omitir → 422 (nunca ADMIN silencioso). Depois, o `default="admin"` da coluna `usuarios.papel` foi removido do mapping (`src/autenticacao/infraestrutura/mapping.py`), tornando a inserção sem papel um fail-CLOSED (NOT NULL) em vez de fail-OPEN. PR #84 (commit c27da5b) + housekeeping #96 (commit 5404826).

14. Refresh token não vale como access + pré-hash bcrypt 72 bytes

- **Motivação:** dois gaps da auditoria — o claim type do JWT não era checado (um refresh token era aceito como access, CWE) e o bcrypt trunca senhas em 72 bytes (colisão de prefixo).
- **Evidência:** TD-028/TD-029. `obter_usuario_atual` (`src/autenticacao/interfaces/middleware.py`) exige `type == "access"` → 401 caso con-

trário; `hash_senha` pré-hasha `base64(sha256(senha))` antes do `bcrypt` (`src/autenticacao/infraestrutura/password_hasher.py`, padrão `bcrypt_sha256`). PR #105 (commit 3bc6c4d).

15. Logout revoga também o refresh token (fecha CWE-613)

- **Motivação:** o logout só revogava o `jti` do access; o refresh sobrevivia, permitindo reemitir um access após o logout.
- **Evidência:** bug da auditoria (issue #118/#121). O logout passou a aceitar o refresh opcional no corpo (`Body(None, embed=True)`, sem quebrar clientes só-header) e revogar ambos os `jti`, best-effort e escopado ao dono; revogar ficou idempotente. Corrigido em c33de8a (PR #142).

16. Seed de admin rejeita a senha demo pública

- **Motivação:** o `ADMIN_PASSWORD` de demonstração (`pytstop-admin-demo-2026`) é commitado em `k8s/secret.yaml` e no `docker-compose`. Sem guarda, um deploy poderia subir produção com essa senha pública.
- **Evidência:** `scripts/seed_admin.py` rejeita o valor `demo` via `frozenset`, com teste de regressão em `tests/unitarios/scripts/test_seed_admin.py`. Issue #95, housekeeping da entrega (commit 5404826).

A.5 Segurança de dados e LGPD (2 extras)

17. Erasure LGPD em cascata para veículos

- **Motivação:** a anonimização do cliente (direito ao esquecimento) deixava a placa do veículo — também PII — intacta e vinculável. A cascata fecha o vazamento residual.
- **Evidência:** issue #72. Migração `006_widen_veiculos_placa.py` alarga a coluna `placa` para acomodar o sentinela `PlacaAnonimizada`; o erasure cascadeia para os veículos vinculados. Também tornado admin-only com trilha de auditoria (#76, commit 064ef1f). PR #112 (commit ea40b2b).

18. Scrub de PII em tracebacks e logs da stdlib

- **Motivação:** mesmo com cifragem em repouso, um traceback ou um access log do uvicorn (com `?documento=CPF` na query) vazaria PII em texto plano.
- **Evidência:** issue #86. Em `src/compartilhado/infraestrutura/logging.py`: `format_exc_info` reordenado para preceder `scrub_pii` (o traceback vira string mascarada); `ProcessorFormatter` instalado no root logger religando os loggers do uvicorn ao scrubber; scrubber ampliado

com padrão de telefone BR e denylist de chaves sensíveis (password/token/secret/...). PR #86 (commit 04217f3).

A.6 Persistência, domínio e concorrência (4 extras)

19. Lock pessimista nas transições da OS + ordem global de locks

- **Motivação:** transições de OS concorrentes sem controle de concorrência podiam emitir N eventos/e-mails e a reserva de estoque sofria lost-update (sobre-venda). Correção de concorrência real.
- **Evidência:** issues #82/#83. obter_por_id(*, com_lock=True) com `SELECT ... FOR UPDATE` nas 11 transições de mutação da OS ([src/ordem_servico/infraestrutura/repository.py](#)); ordem global OS→Estoque + locks multi-item em ordem de item_estoque_id previnem dead-lock. Reforçado com `populate_existing=True` (evita instância stale do identity map, #117). Testes de concorrência com Postgres real em `tests/integracao/ordem_servico/test_concorrencia_lock.py`. PR #101 (commit 7b58156) + c33de8a.

20. Snapshot do escopo aprovado do orçamento (JSONB) — cobrança correta do complementar

- **Motivação:** gerar_orcamento_complementar sobrescrevia o orçamento e a rejeição não revertia nada — a OS acabava cobrando trabalho recusado e mantinha reservas de estoque nunca liberadas.
- **Evidência:** issues #111/#122. Snapshot do escopo aprovado (orçamento + ids dos itens cobertos) congelado nas aprovações e persistido na coluna JSONB `escopo_aprovado_json` (migração [007_escopo_aprovado_os.py](#), reversível); a rejeição do complementar restaura o orçamento aprovado, remove itens fora do escopo e libera as reservas na mesma UoW; finalizar_servico recusa itens fora do escopo. Commit e794bfc.

21. orcamento_json migrado de Text para jsonb nativo

- **Motivação:** o orçamento vivia numa coluna Text com serialização manual `json.dumps/json.loads` no mapping — frágil e fora do padrão da `outbox.payload`.
- **Evidência:** TD-005. Migração [004_orcamento_jsonb.py](#) troca para jsonb, removendo a camada manual no [src/ordem_servico/](#)

[infraestrutura/mapping.py](#) (`JSONB().with_variant(JSON(), "sqlite")`). PR #68 (commit c363369).

22. Value Object Contato (substitui primitivo str)

- **Motivação:** o contato do cliente era um primitivo str sem validação nem representação PII-safe — débito tático de DDD herdado da fase 1.
- **Evidência:** TD-007, [src/cliente_veiculo/dominio/contato.py](#) — texto livre validado (não-vazio, `<=255`, `strip`, `__repr__` PII-safe), persistido na mesma coluna via shadow + event listeners (padrão CPF/Placa, sem migração). PR #70 (commit efd5431).

A.7 Infraestrutura, escala e DX (4 extras)

23. Rate limiter com storage compartilhado (Redis) sob HPA

- **Motivação:** o rate limiter in-memory por pod diverge entre réplicas — sob HPA, cada réplica conta separadamente e o limite global fica incorreto.
- **Evidência:** TD-016, [ADR-023](#). `slowapi` com `storage_uri` apontando para Redis (env `RATE_LIMIT_STORAGE_URI`), Deployment+Service em [k8s/redis.yaml](#), com degradação graciosa para per-réplica se o Redis cair. PR #62 (commit 70b70e3).

24. Rate-limit pelo IP real do cliente atrás de proxy confiável

- **Motivação:** atrás de ingress, o rate-limit por IP do peer imediato colapsa todo o tráfego externo num único bucket. É preciso ler o `X-Forwarded-For` confiável.
- **Evidência:** TD-023, [ADR-023](#). `ProxyHeadersMiddleware` do `uvicorn` aplicado quando `TRUSTED_PROXIES` está configurado ([src/compartilhado/interfaces/middleware.py](#)), reescrevendo `request.client` a partir do `XFF`; default vazio (não confia em `XFF`, sem spoof). PR #67 (commit 7bb9837).

25. Migração em Job dedicado antes do rollout (fim da corrida multi-réplica)

- **Motivação:** rodar o Alembic no endpoint do pod cria corrida com N réplicas subindo em paralelo. Um Job dedicado, aplicado antes do rollout, serializa a migração.
- **Evidência:** TD-015, [ADR-019](#). [k8s/jobs/migration-job.yaml](#) (`pytstop-migrate`) aplicado com `kubectl wait --for=condition=complete`;

RUN_MIGRATIONS_ON_STARTUP/RUN_SEED_ON_STARTUP passam a false no cluster. PR #64 (commit 02900d4).

26. Índice B-tree em `itens_da_ordem.item_estoque_id`

- **Motivação:** a checagem `existe_ativa_com_item_estoque` fazia full scan sem índice de suporte — degrada com o volume de itens.
- **Evidência:** TD-025. Migração `005_indice_item_estoque.py` (reversível) cria `ix_itens_da_ordem_item_estoque_id`; a declaração Index no mapping mantém o `create_all` em sync. EXPLAIN confirma Index Scan. PR #107 (commit dbc17db).

A.8 Arquitetura, qualidade e plataforma (3 extras)

27. Contrato de camadas verificado por import-linter (Clean Architecture executável)

- **Motivação:** o enunciado pede Clean Architecture, mas não sua verificação automática. Sem gate, a separação de camadas degrada silenciosamente.
- **Evidência:** TD-019 (RNF-017), [ADR-015](#). Contratos em `[tool.importlinter] (pyproject.toml)` verificados por `make lint-arch` na CI; o contrato `forbidden` passou a proibir `aplicacao` → infraestrutura em **todos** os contextos após a extração de `PasswordHasherPort/JWTServicePort` na autenticação. PR #50 (commit incluído no histórico de fechamento).

28. Cobertura ≥95% com gate explícito no CI

- **Motivação:** o enunciado pede apenas testes dos fluxos críticos. O grupo manteve gate de cobertura muito acima do mínimo (a fase 1 exigia 80%), tornando-o explícito no pipeline.
- **Evidência:** TD-031. `--cov-fail-under=95` explícito no step de cobertura de `src/` no `.github/workflows/ci.yml`, alinhado ao `.coveragerc`; cobertura de ~95,3% confirmada na CI da main. PR #106 (commit 1023e45).

29. Migração para Python 3.14 (runtime, tooling e imagens alinhados)

- **Motivação:** manter o runtime atualizado (o desbloqueio veio com o NiceGUI 3, que removeu a dependência `vbui ld`). Não exigido pela fase.

- **Evidência:** `.python-version=3.14` + Dockerfile builder `ghcr.io/astral-sh/uv:python3.14-bookworm-slim` e runtime `python:3.14-slim` (mesma minor, senão o venv copiado quebra) + `ui/Dockerfile` + os 7 pins `python-version: "3.14"` nos workflows. Todas as deps com wheel cp314; teste de mesa no app 3.14.6 vivo. PR #150 (commit f86826f).

Total

29 funcionalidades extras, todas verificadas contra o repositório, distribuídas em:

Tema	Extras
Entrega durável de eventos e resiliência	4 (itens 1–4)
Observabilidade	3 (itens 5–7)
Segurança de infraestrutura e supply chain	5 (itens 8–12)
Hardening de autenticação	4 (itens 13–16)
Segurança de dados e LGPD	2 (itens 17–18)
Persistência, domínio e concorrência	4 (itens 19–22)
Infraestrutura, escala e DX	4 (itens 23–26)
Arquitetura, qualidade e plataforma	3 (itens 27–29)

Nenhum desses itens era exigido pela Fase 2. São iniciativa de qualidade do grupo, materializando a Boy Scout Rule registrada no ledger de dívida técnica: cada evolução deixa o código e a documentação melhores do que os encontrou. O conjunto completo dos 26 itens de dívida técnica resolvidos está no ledger; os itens de maior valor também aparecem na seção 6 do documento de entrega.